

Application Cloud de Calcul de l'IMC

Documentation détaillée

Travaux Pratique 1
Question I

8CLD876 — Conception et architecture des systèmes d'infonuagique

I - Introduction et description du système

1.1. Objectif

Concevoir et développer une application d'infonuagique permettant de calculer l'indice de masse corporelle (IMC) d'une personne, de le classer selon les seuils de l'Organisation mondiale de la santé, et de conserver l'historique des mesures dans une base de données.

1.2. Fonctionnalités

L'utilisateur saisit un poids et une taille dans une interface web. Le système calcule l'IMC selon la formule de Quetelet, détermine la catégorie correspondante, enregistre la mesure et affiche l'historique des vingt derniers calculs.

$$IMC = \frac{M}{T^2}$$

Où M représente son masse (ou le poids) en kg et T la taille en mètres.

Voici les seuils considérés par l'OMS :

IMC	Catégorie
moins de 18,5	Insuffisance pondérale
18,5 – 24,9	Corpulence normale
25,0 – 29,9	Surpoids
30,0 – 34,9	Obésité modérée (classe I)
35,0 – 39,9	Obésité sévère (classe II)
40,0 et plus	Obésité morbide (classe III)

1.3. Technologies retenues

Couche	Technologie	Justification
Interface	HTML, CSS, JavaScript	Aucune dépendance, exécution dans le navigateur
Serveur web	nginx 1.27 alpine	Léger, et sert aussi de proxy inverse
Logique métier	Python 3.12, Flask, Gunicorn	Rapidité de développement, Gunicorn pour la production
Base de données	MySQL 8.0	Relationnelle, Imposée par la nature tabulaire des données
Conteneurisation	Docker, Docker Compose	Isolation, reproductibilité, orchestration locale

1.4. Accès à l'application

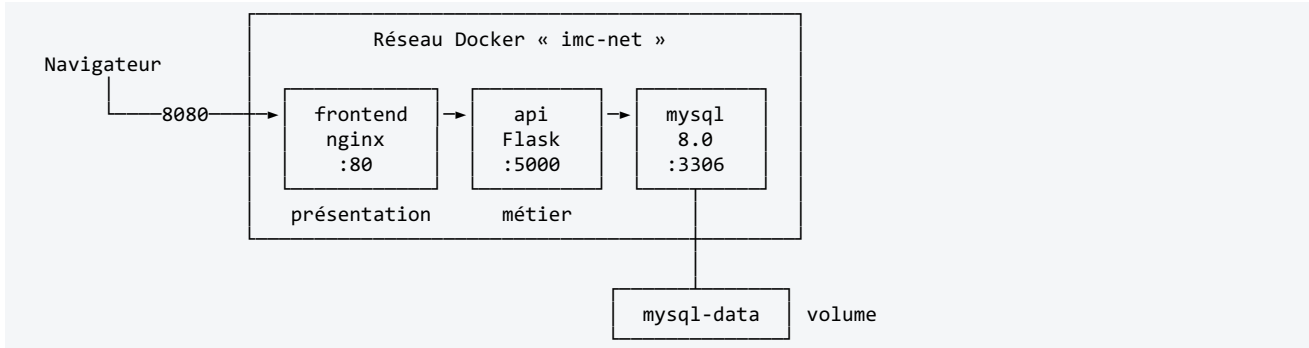
L'application est accessible à l'adresse <http://localhost:8080> après exécution de la command `docker compose up -d --build` depuis le chemin d'accès du dossier après l'avoir téléchargé. Elle s'exécute localement, sur toute machine disposant de Docker, indépendamment du système d'exploitation.

NB : Ce projet est aussi posté sur le GitHub personnel de Noureldin: <https://github.com/Nour-mah/imc-app> (le fichier envoyé sur moodle est la dernière version du repository avant la date de rendu)

II - Architecture de l'application

2.1. Vue d'ensemble

L'application suit une architecture **trois tiers**, chaque tiers étant isolé dans son propre conteneur et relié aux autres par un réseau Docker privé.



Tiers	Service	Conteneur	Rôle
Présentation	frontend	imc-frontend	sert la page web et relaie /api vers l'API
Métier	api	imc-api	calcule, classe, valide, dialogue avec la base
Données	mysql	imc-mysql	persiste les mesures dans la table mesures

2.2. Cheminement d'une requête

1. Le navigateur demande `http://localhost:8080/api/imc`.
2. Le port 8080 de la machine hôte est redirigé vers le port 80 du conteneur frontend.
3. nginx reconnaît le préfixe `/api/` et transmet la requête à `http://api:5000` — le nom api étant résolu par le DNS interne du réseau Docker.
4. L'API calcule l'IMC, le classe, puis ouvre une connexion vers l'hôte mysql, résolu de la même manière.
5. MySQL écrit la mesure dans `/var/lib/mysql`, dossier monté sur le volume mysql-data.

Aucune adresse IP n'apparaît dans le code, la configuration nginx ou le fichier d'orchestration : toute la communication repose sur des noms de services.

2.3. Décisions d'architecture

- **Un seul port publié**, celui du frontend : l'API et la base ne sont joignables que depuis le réseau interne.
- **Proxy inverse** : le navigateur ne voit qu'une seule origine, ce qui supprime toute problématique de CORS.
- **Découverte par nom de service** : un conteneur peut être recréé avec une autre adresse IP sans modifier la configuration.
- **Configuration externalisée** par variables d'environnement : la même image fonctionne en local et dans le réseau Docker.

- **Démarrage ordonné** par test de santé : l'API attend que MySQL accepte réellement les connexions.
- **État séparé du calcul** : les données vivent dans un volume nommé, les conteneurs restent jetables.

III - Guide de l'utilisateur

3.1. Prérequis

- Docker Desktop (Docker Engine 20.10 ou supérieur, Compose v2)
- Le port 8080 disponible

3.2. Installation

```
git clone https://github.com/Nour-mah/imc-app.git
cd imc-app
docker compose up -d --build
```

(on peut aussi remplacer la première commande par le fait de télécharger le fichier depuis moodle et le poser dans le pwd)

Le premier lancement télécharge l'image MySQL, construit les images de l'API et du frontend, crée la base et exécute le schéma. Compter environ une minute.

Vérification :

```
docker compose ps
```

Les trois services doivent apparaître, imc-mysql portant la mention (healthy).

3.3. Utilisation

6. Ouvrir <http://localhost:8080>.
7. Saisir un poids en kilogrammes et une taille en mètres.
8. Cliquer sur Calculer.
9. Le résultat s'affiche sous le formulaire : valeur de l'IMC et catégorie.
10. La mesure est enregistrée et apparaît en tête du tableau Historique, accompagnée de sa date.

En cas de saisie invalide, un message explicite s'affiche en rouge.

3.4. Arrêt

```
docker compose down
```

Les conteneurs sont supprimés, les données conservées. L'option -v supprime également le volume, donc l'ensemble des mesures.

3.5. Interface de programmation

Méthode	Route	Corps	Réponse
GET	/health	—	200 — état du service
POST	/api/imc	{"poids": 70, "taille": 1.75}	201 — IMC, catégorie, identifiant
GET	/api/historique	—	200 — les vingt dernières

			mesures
--	--	--	---------

IV. Tests et exécution

Les résultats bruts figurent dans le fichier docs/02-tests-execution.txt, produit par exécution réelle le 2 octobre 2026.

4.1. État des services

NAME	IMAGE	SERVICE	STATUS	PORTS
imc-api	imc-api:1.0	api	Up	5000/tcp
imc-frontend	imc-frontend:1.0	frontend	Up	0.0.0.0:8080->80/tcp
imc-mysql	mysql:8.0	mysql	Up (healthy)	3306/tcp, 33060/tcp

Deux observations : imc-mysql est bien marqué healthy, ce qui confirme le fonctionnement du test de santé ; et l'API n'expose aucun port vers l'hôte, conformément au principe du point d'entrée unique.

4.2. Calculs valides

Entrée	IMC obtenu	Catégorie	Code
70 kg / 1,75 m	22,86	Corpulence normale	201
92 kg / 1,80 m	28,40	Surpoids	201
48 kg / 1,72 m	16,22	Insuffisance pondérale	201
105 kg / 1,78 m	33,14	Obésité modérée (classe I)	201

Vérification manuelle du premier cas : $70 \div (1,75 \times 1,75) = 22,857...$ soit 22,86 après arrondi à deux décimales.

4.3 Validation des entrées

Entrée	Code	Message retourné
{"poids":70,"taille":0}	400	Taille doit etre superieure a 0
{"poids":70}	400	Requete invalide, poids et taille requis
{"poids":"abc","taille":1.75}	400	Poids et taille doivent etre des nombres
{"poids":-5,"taille":1.75}	400	Poids doit etre superieur a 0
{"poids":70,"taille":175}	400	La taille doit etre en metres, pas en centimetres

Aucune entrée invalide ne provoque d'erreur 500. Le dernier cas mérite une justification : sans ce contrôle, l'API répondrait 201 avec un IMC de 0,02 et la catégorie « Insuffisance pondérale ». Une réponse fausse mais vraisemblable est plus dangereuse qu'une erreur explicite.

4.4. Persistance des données

```
Avant : total = 15 mesures
$ docker compose down → les trois conteneurs sont supprimés
$ docker compose up -d → les trois conteneurs sont recréés
Après : total = 15 mesures
```

Les conteneurs ont été détruits puis recréés à partir des images ; les données sont intactes. Elles résident dans le volume imc-app-repo_mysql-data, indépendant du cycle de vie des conteneurs.

4.5. Découverte de services

```
$ docker compose exec frontend ping -c 2 api
PING api (172.21.0.3): 56 data bytes
64 bytes from 172.21.0.3: seq=0 ttl=64 time=0.162 ms
```

Le nom api est résolu vers l'adresse du conteneur par le DNS interne du réseau imc-net.

4.6. Contenu de la base

id	poids	taille	imc	categorie	date_creation
15	92.00	1.80	28.40	Surpoids	2026-10-02 20:07:34
14	70.00	1.75	22.86	Corpulence normale	2026-10-02 20:07:34
12	105.00	1.78	33.14	Obesite moderee (classe I)	2026-10-02 20:06:59
11	48.00	1.72	16.22	Insuffisance ponderale	2026-10-02 20:06:59

Annexe A — Structure du projet

```
imc-app/
├── api/
│   ├── app.py           API Flask : routes, validation, accès MySQL
│   ├── requirements.txt  dépendances, versions figées
│   └── Dockerfile        image Python 3.12-slim, exécutée par Gunicorn
├── db/
│   └── init.sql          schéma de la table mesures
├── frontend/
│   ├── index.html       formulaire, résultat, historique
│   ├── app.js           appels à l'API en JSON
│   ├── default.conf     configuration nginx : statique et proxy inverse
│   └── Dockerfile        image nginx alpine
├── docker-compose.yml   orchestration des trois services
├── .dockerignore
└── README.md
```

Code source complet : <https://github.com/Nour-mah/imc-app>

Annexe B — Capture de l'application

Calculateur d'indice de masse corporelle

Poids (kg) 70 Taille (m) 1.75 Calculer

IMC : 22.86 — Corpulence normale

Historique

Date	Poids	Taille	IMC	Categorie
2026-10-02 20:07:07	70	1.75	22.86	Corpulence normale
2026-10-02 20:06:59	105	1.78	33.14	Obesite moderee (classe I)
2026-10-02 20:06:59	48	1.72	16.22	Insuffisance ponderale
2026-10-02 20:06:59	92	1.8	28.4	Surpoids
2026-10-02 20:06:59	70	1.75	22.86	Corpulence normale
2026-10-02 19:42:17	80	1.8	24.69	Corpulence normale
2026-10-02 19:42:15	81	1.8	25	Surpoids
2026-10-02 19:42:13	82	1.8	25.31	Surpoids
2026-10-02 19:42:09	83	1.8	25.62	Surpoids
2026-10-02 19:42:05	80	1.8	24.69	Corpulence normale
2026-10-02 19:41:54	85	1.8	26.23	Surpoids
2026-10-02 19:39:15	95	1.8	29.32	Surpoids
2026-10-02 19:39:05	106	1.8	32.72	Obesite moderee (classe I)

Interface de l'application après un calcul (70 kg, 1,75 m) : le résultat s'affiche sous le formulaire et la mesure apparaît en tête de l'historique, relu depuis la base de données.